HUNT-SAGE · THREAT HUNTING CHEAT SHEET

KQL for Hunters

Operators, functions, table maps and ready-to-run hunting patterns for Microsoft Sentinel and Defender XDR advanced hunting.

github.com/iamjammie/hunt-sage · iamjammie.github.io/hunt-sage · Built 2026-11-03

Anatomy

```
let lookback = 7d;                                  // let binds scalars, tables or functions
DeviceProcessEvents                                 // table (Defender XDR: Timestamp; Sentinel: TimeGenerated)
| where Timestamp > ago(lookback)                   // time filter FIRST: the biggest performance win
| where FileName has "powershell"                   // 'has' is term-indexed; 'contains' is not
| extend Cmd = tolower(ProcessCommandLine)          // add or overwrite columns
| summarize Count = count(), Hosts = dcount(DeviceName) by Cmd, bin(Timestamp, 1h)
| order by Count desc                               // 'order by' == 'sort by'
| take 50                                           // == 'limit'
```

- Case: operators ending in `~` and `has` / `contains` are case-insensitive; `==`, `contains_cs`, `has_cs` are case-sensitive. Column and table names are case-sensitive.
- Strings: `"double"` or `'single'` quotes; `@"raw\string"` for paths and regex; `strcat()` to concatenate.
- Dynamic (JSON) columns: `AdditionalFields.Field`, `AdditionalFields["Field"]`, `todynamic(s)`, `parse_json(s)`; cast before comparing (`tostring()`, `toint()`).
- Comments: `//` to end of line. Multi-statement queries separate `let` bindings with `;`.
- Sentinel and XDR share the language; the differences are table names, the time column and retention (see the maps below).

String operators

Operator	Meaning	Performance / notes
`has`, `!has`	whole term present, case-insensitive	**Fastest**: uses the term index. Will not match partial tokens (`has "power"` misses `powershell`)
`has_any (list)`, `has_all (list)`	any / all terms present	Same index; pass a `dynamic([...])` or a `let` list
`has_cs`	whole term, case-sensitive	Indexed
`contains`, `!contains`, `contains_cs`	substring anywhere	Full scan of the column: filter time and table first
`startswith`, `endswith` (+ `_cs`)	prefix / suffix	Moderate; `endswith @"\lsass.exe"` is a common pattern
`in`, `!in`, `in~`, `!in~`	set membership (`in~` case-insensitive)	Up to 1,000,000 values; prefer over chained `or`
`==`, `!=`, `=~`, `!~`	equality (`=~` case-insensitive)	`=~` for file and account names, which vary in case
`matches regex`	RE2 regex	Slowest; `(?i)` for case-insensitive; anchor where you can
`has_any_ipv4`, `ipv4_is_in_range`, `ipv4_is_private`	IP helpers	`ipv4_is_private(RemoteIP)` separates east-west from internet

Time

Expression	Meaning
`ago(7d)`, `ago(90m)`	now minus a timespan (`d h m s ms`)
`between (ago(30d) .. ago(1d))`	inclusive range; also works on numbers
`bin(Timestamp, 15m)`	floor to bucket (alias `floor`)
`bin_at(Timestamp, 1d, datetime(2026-01-01))`	buckets aligned to a reference point
`startofday(t)`, `startofweek(t)`, `endofmonth(t)`	boundaries
`datetime_diff('second', t2, t1)`	signed difference; also `'minute'`, `'hour'`, `'day'`
`format_datetime(t, "yyyy-MM-dd HH:mm")`	string output

Expression	Meaning
`make_datetime(2026, 11, 3, 8, 0)`	build a datetime
`todatetime("2026-11-03T08:00:00Z")`	parse; ISO 8601 safest
`datetime_part("hour", t)` , `dayofweek(t)`	components; `dayofweek` returns a timespan (`1d` = Monday)

Everything is **UTC**. Convert only for display: `datetime_utc_to_local(t, "Asia/Kolkata")` . Working-hours logic in a hunt should use UTC offsets explicitly.

Aggregation toolbox

Function	Use
`count()` , `countif(cond)`	rows / rows matching
`dcount(x)` , `dcountif(x, cond)`	distinct count. Accuracy is an estimate by default; `dcount(x, 4)` for high accuracy on small sets
`make_set(x, 50)` , `make_list(x, 50)`	distinct / all values as an array; **always cap** (default 1,048,576)
`make_set_if(x, cond, 20)`	conditional set
`arg_max(Timestamp, *)` , `arg_min(Timestamp, Col)`	the whole row at the max / min: "latest state per key"
`min()` , `max()` , `sum()` , `sumif()`	numeric
`avg()` , `stdev()` , `variance()` , `percentiles(x, 50, 95, 99)`	distribution; `percentile(x, 95)` for one
`take_any(x)` (older: `any()`)	a sample value, cheap
`hll(x)` , `dcount_hll(agg)`	mergeable distinct counts for summary tables
`count_distinct(x)`	exact distinct count (newer, heavier than `dcount`)

```
// stack then read the long tail: the general hunting shape
DeviceProcessEvents
| where Timestamp > ago(30d)
| summarize Hosts = dcount(DeviceName), Count = count(), First = min(Timestamp), Sample = take_any(ProcessCommandLin
e)
          by Parent = tolower(InitiatingProcessFileName), Child = tolower(FileName)
| where Hosts <= 2
| order by Hosts asc, Count asc
```

Joins and lookups

Kind	Rows kept	Hunting use
`kind=innerunique` (**default**)	left side deduplicated on the key first	**Trap**: silently drops left rows; almost never what you want in a hunt
`kind=inner`	all matches	process to network, logon to process
`kind=leftouter`	all left + matches	enrich without losing rows
`kind=leftanti`	left rows with **no** match	"never seen before", missing baseline
`kind=rightanti` / `rightouter`	mirror of the above	
`kind=fullouter`	everything	reconciliation
`lookup`	left join against a small dimension table	faster than `join` for enrichment (asset lists, watchlists)

```
let baseline = DeviceProcessEvents | where Timestamp between (ago(30d) .. ago(1d)) | distinct DeviceName, FileName;
DeviceProcessEvents
| where Timestamp > ago(1d)
| join kind=leftanti baseline on DeviceName, FileName      // first-seen process per host today
| summarize Count = count(), Cmd = take_any(ProcessCommandLine) by DeviceName, FileName
```

- Filter and `summarize` both sides **before** joining; put the smaller table on the left (or `hint.strategy=broadcast` when the left is small).
- Join keys must match type and case; use `$left.Col == $right.OtherCol` when names differ.

- `union TableA, TableB` (add `isfuzzy=true` so a missing table does not fail the query); `union withsource=Src Device*` to tag the origin.

Shaping

Operator / function	Does
`project a, b`, `project-away x`, `project-rename New = Old`, `project-reorder`	choose, drop, rename, order columns
`extend New = expr`	add columns
`parse Col with * "user=" User " " *`	positional string parsing; `parse-where` drops non-matches; `parse kind=regex`
`extract(@"-enc\s+(\S+)", 1, Cmd)`, `extract_all(@"(\d+\.\d+\.\d+\.\d+)", Cmd)`	regex capture (one / all)
`split(Path, @"\")`, `tostring(split(x, ",")[0])`	split to array; index then cast
`parse_json(s)` / `todynamic(s)`	string to dynamic
`mv-expand Col`	one row per array element; `mv-expand Col to typeof(string)`
`mv-apply x = Col on (where x.k == 1)`	sub-query per array element
`bag_unpack(Dyn)` (via `evaluate`)	dynamic bag to columns
`pack_array()`, `pack()` / `bag_pack()`	build arrays / bags
`tostring`, `toint`, `tolong`, `todouble`, `tobool`, `tohex`	casts (`toint` of a bad string returns null, not an error)
`coalesce(a, b)`, `iff(cond, a, b)`, `case(c1, v1, c2, v2, default)`	null handling and branching
`strcat()`, `strcat_delim()`, `strlen()`, `substring()`, `trim(@"\s+", s)`, `tolower()`, `toupper()`	strings
`replace_string(s, "a", "b")`, `replace_regex(s, @"\d+", "N")`	replace
`url_decode()`, `base64_decode_tostring()`, `base64_decode_toarray()`	decoding (UTF-16LE PowerShell `-enc` needs the array form, see below)
`countof(s, "sub")`, `countof(s, @"[0-9]", "regex")`	occurrence count
`format_bytes(n)`, `format_timespan()`	display
`column_ifexists("Col", "")`	tolerate schema differences across tables
`distinct`, `top 20 by Count`, `sample 100`, `sample-distinct`	reduce

```
// decode -EncodedCommand (UTF-16LE): decode to bytes, drop the zero bytes, make a string
DeviceProcessEvents
| where ProcessCommandLine matches regex @"(?i)\s-(e|ec|enc|encodedcommand)\s+[A-Za-z0-9+/=]{20,}"
| extend B64 = extract(@"(?i)\s-(?:e|ec|enc|encodedcommand)\s+([A-Za-z0-9+/=]+)", 1, ProcessCommandLine)
| extend Decoded = replace_regex(base64_decode_tostring(B64), @"\x00", "")
| project Timestamp, DeviceName, AccountName, Decoded
```

Series, sequences and anomalies

Tool	Use
`make-series Count = count() on Timestamp from ago(14d) to now() step 1h by Key`	gap-free time series per key
`series_decompose_anomalies(Series, 1.5)`	seasonal + trend decomposition, returns anomaly flags
`series_stats(Series)`, `series_outliers(Series)`	min/max/avg/variance; Tukey outlier scores
`render timechart` / `render columnchart`	visual in the portal (ignored by scheduled rules)
`serialize`, `prev(Col)`, `next(Col)`, `row_number()`	ordered-window functions; sort first
`row_window_session(Timestamp, 1h, 5m, Key != prev(Key))`	sessionise events
`scan` operator	state-machine matching of event sequences (A then B within a window)
`top-nested`	nested top-N (top hosts, then top processes per host)

```
// beaconing: regular intervals to the same destination
DeviceNetworkEvents
| where Timestamp > ago(1d) and RemoteIPType == "Public" and ActionType == "ConnectionSuccess"
| sort by DeviceName, RemoteIP, Timestamp asc
| serialize
| extend Delta = datetime_diff('second', Timestamp, prev(Timestamp))
| where prev(DeviceName) == DeviceName and prev(RemoteIP) == RemoteIP
| summarize N = count(), AvgGap = avg(Delta), Sd = stdev(Delta), Procs = make_set(InitiatingProcessFileName, 5) by D
eviceName, RemoteIP, RemoteUrl
| where N >= 20 and AvgGap > 30 and Sd < 0.2 * AvgGap        // low jitter = machine-driven
| order by N desc
```

Hunting patterns

```
// 1. Long or suspicious command lines (length plus a scripting parent)
DeviceProcessEvents
| where Timestamp > ago(7d) and strlen(ProcessCommandLine) > 1000
| where InitiatingProcessFileName in~ ("winword.exe", "excel.exe", "outlook.exe", "wscript.exe", "mshta.exe", "cmd.e
xe")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, FileName, ProcessCommandLine
```

```
// 2. DNS: long subdomains and high label counts (tunnelling leads)
DeviceNetworkEvents
| where Timestamp > ago(1d) and ActionType == "DnsConnectionInspected"
| extend Q = tostring(parse_json(AdditionalFields).query)
| extend Labels = countof(Q, "."), Len = strlen(Q)
| where Len > 60 or Labels > 5
| summarize Queries = count(), Sample = take_any(Q) by DeviceName, Domain = strcat_array(array_slice(split(Q, "."),
-2, -1), ".")
| order by Queries desc
```

```
// 3. Rare external destinations from servers (first seen in 1 d, absent from 30 d)
let servers = DeviceInfo | where OSPlatform has "Server" | distinct DeviceName;
let known = DeviceNetworkEvents | where Timestamp between (ago(30d) .. ago(1d)) and RemoteIPType == "Public" | disti
nct RemoteIP;
DeviceNetworkEvents
| where Timestamp > ago(1d) and RemoteIPType == "Public" and DeviceName in (servers)
| join kind=leftanti known on RemoteIP
| summarize Conns = count(), Procs = make_set(InitiatingProcessFileName, 5) by DeviceName, RemoteIP, RemotePort
```

```
// 4. Logon type 3 fan-out (lateral movement): one account to many hosts in an hour
DeviceLogonEvents
| where Timestamp > ago(1d) and LogonType == "Network" and ActionType == "LogonSuccess"
| where not(AccountName endswith "$")
| summarize Hosts = dcount(DeviceName), HostList = make_set(DeviceName, 20) by AccountName, bin(Timestamp, 1h)
| where Hosts >= 10
// Sentinel equivalent: SecurityEvent | where EventID == 4624 and LogonType == 3 | summarize dcount(Computer) by Tar
getUserName, bin(TimeGenerated, 1h)
```

```
// 5. New local administrators (Security 4732 into Administrators)
SecurityEvent
| where TimeGenerated > ago(7d) and EventID == 4732 and TargetUserName =~ "Administrators"
| project TimeGenerated, Computer, Actor = SubjectUserName, MemberName, MemberSid
```

```
// 6. LSASS access from unexpected processes (Defender ProcessAccess / OpenProcess telemetry)
DeviceEvents
| where Timestamp > ago(7d) and ActionType in ("OpenProcessApiCall", "ProcessAccess")
| where FileName =~ "lsass.exe"
| where InitiatingProcessFolderPath !startswith @"c:\windows\system32" and InitiatingProcessFolderPath !startswith
@"c:\program files"
| summarize Hosts = dcount(DeviceName), Count = count() by InitiatingProcessFileName, InitiatingProcessFolderPath, I
nitiatingProcessSHA256
| order by Hosts asc
```

```
// 7. Registry Run-key stacking across the fleet
DeviceRegistryEvents
| where Timestamp > ago(30d) and ActionType in ("RegistryValueSet", "RegistryKeyCreated")
| where RegistryKey has_any (@"\CurrentVersion\Run", @"\CurrentVersion\RunOnce", @"\Winlogon", @"\Image File Executi
on Options")
| summarize Hosts = dcount(DeviceName), First = min(Timestamp), Writer = take_any(InitiatingProcessFileName) by Regi
stryValueName, RegistryValueData
| where Hosts <= 3
```

```
// 8. Sign-in from a new country followed by MFA failures (Entra ID)
let known = SigninLogs | where TimeGenerated between (ago(30d) .. ago(1d)) and ResultType == 0 | distinct UserPrinci
palName, Country = tostring(LocationDetails.countryOrRegion);
SigninLogs
| where TimeGenerated > ago(1d)
| extend Country = tostring(LocationDetails.countryOrRegion)
| join kind=leftanti known on UserPrincipalName, Country
| summarize Attempts = count(), MfaFailed = countif(ResultType in ("500121", "50074", "50076")), Success = countif(R
esultType == 0),
            Methods = make_set(tostring(AuthenticationDetails[0].authenticationMethod), 5) by UserPrincipalName, Cou
ntry, IPAddress
| where Success > 0 or MfaFailed >= 3
```

```
// 9. Impossible travel with prev(): same user, two countries, < 2 h apart
SigninLogs
| where TimeGenerated > ago(1d) and ResultType == 0
| extend Country = tostring(LocationDetails.countryOrRegion)
| sort by UserPrincipalName, TimeGenerated asc
| serialize
| extend PrevCountry = prev(Country), PrevTime = prev(TimeGenerated), PrevUser = prev(UserPrincipalName)
| where PrevUser == UserPrincipalName and PrevCountry != Country and datetime_diff('minute', TimeGenerated, PrevTim
e) < 120
| project TimeGenerated, UserPrincipalName, PrevCountry, Country, IPAddress, AppDisplayName
```

```
// 10. OAuth consent grants to mail-reading scopes
AuditLogs
| where TimeGenerated > ago(7d) and OperationName in ("Consent to application", "Add delegated permission grant")
| mv-expand TargetResources
| extend App = tostring(TargetResources.displayName), Props = TargetResources.modifiedProperties
| mv-expand Props
| where tostring(Props.newValue) has_any ("Mail.Read", "Mail.ReadWrite", "MailboxSettings", "offline_access", "File
s.ReadWrite.All")
| project TimeGenerated, Actor = tostring(InitiatedBy.user.userPrincipalName), App, Scope = tostring(Props.newValue)
```

```
// 11. Inbox rules that hide mail (BEC pattern)
OfficeActivity
| where TimeGenerated > ago(7d) and Operation in ("New-InboxRule", "Set-InboxRule")
| extend P = parse_json(Parameters)
| mv-apply P on (summarize Params = make_bag(bag_pack(tostring(P.Name), tostring(P.Value))))
| where Params has_any ("DeleteMessage", "MoveToFolder", "ForwardTo", "RedirectTo", "MarkAsRead")
| project TimeGenerated, UserId, ClientIP, Operation, Params
```

```
// 12. Process to network join: what talked out, and what launched it
DeviceNetworkEvents
| where Timestamp > ago(1d) and RemoteIPType == "Public"
| join kind=inner (DeviceProcessEvents | where Timestamp > ago(1d) | project DeviceId, ProcessId, ProcessCommandLin
e, InitiatingProcessFileName)
      on DeviceId, $left.InitiatingProcessId == $right.ProcessId
| summarize Dests = dcount(RemoteIP), Sample = take_any(RemoteUrl) by DeviceName, FileName = InitiatingProcessFileNa
me, ProcessCommandLine, Parent = InitiatingProcessFileName1
```

Defender XDR tables (advanced hunting)

Table	What it holds	Hunting note
`DeviceProcessEvents`	process creation, command line, hashes, parent, token elevation	the core table
`DeviceNetworkEvents`	connections, listens, DNS inspection (`DnsConnectionInspected`)	`RemoteIPType == "Public"`
`DeviceFileEvents`	file create/modify/delete/rename, `FileOriginUrl` , `IsAzureInfoProtectionApplied`	download origin and MOTW
`DeviceRegistryEvents`	key/value set, create, delete, rename	persistence
`DeviceLogonEvents`	logons by type incl. RemoteInteractive, Network, Batch, Service	`IsLocalAdmin`
`DeviceImageLoadEvents`	DLL loads (sampled on some tiers)	side-loading
`DeviceEvents`	everything else by `ActionType` : `OpenProcessApiCall` , `ProcessAccess` , `NamedPipeEvent` , `WmiBindEventFilterToConsumer` , `AntivirusDetection` , `PowerShellCommand` , `AsrLsassCredentialTheftBlocked` , `ScheduledTaskCreated` , `ServiceInstalled` , `UsbDriveMounted` , `LdapSearch` , `SmartScreenUrlWarning`	`DeviceEvents	distinct ActionType` to discover

Table	What it holds	Hunting note
`DeviceInfo`, `DeviceNetworkInfo`	inventory, OS, IPs, `ExposureLevel`	joins for scoping
`DeviceTvmSoftwareVulnerabilities`, `DeviceTvmSecureConfigurationAssessment`	vulnerability and config posture	pair with hunts
`IdentityLogonEvents`, `IdentityQueryEvents`, `IdentityDirectoryEvents`	Defender for Identity: AD logons, LDAP/SAMR queries, AD object changes	Kerberoasting, BloodHound, DCSync
`EmailEvents`, `EmailAttachmentInfo`, `EmailUrlInfo`, `EmailPostDeliveryEvents`, `UrlClickEvents`	mail flow, attachments, URLs, post-delivery actions, Safe Links clicks	phishing
`CloudAppEvents`	Defender for Cloud Apps and Microsoft 365 activity (`ActionType`, `RawEventData`)	inbox rules, SharePoint downloads
`AlertInfo`, `AlertEvidence`	alerts and their entities	start from an alert, hunt around it
`ExposureGraphNodes`, `ExposureGraphEdges`	Exposure Management graph	attack paths, `make-graph`
`BehaviorInfo`, `BehaviorEntities`	behaviours (lower-fidelity than alerts)	leads

Microsoft Sentinel tables and ASIM

Table	Source
`SecurityEvent`	Windows Security via AMA/Security Events connector (4624, 4688, 4769...)
`Event`	other Windows channels (Sysmon, PowerShell, Task Scheduler) via AMA data collection rules
`Syslog`, `CommonSecurityLog`	Linux syslog; CEF (firewalls, proxies, NDR)
`DnsEvents` / `ASimDnsActivityLogs`	DNS server / normalised DNS
`SigninLogs`, `AADNonInteractiveUserSignInLogs`, `AADServicePrincipalSignInLogs`, `AuditLogs`, `AADRiskyUsers`	Entra ID
`OfficeActivity`	Microsoft 365 Unified Audit Log
`AzureActivity`, `AzureDiagnostics`, `AKSAudit`	Azure control plane and resource logs
`AWSCloudTrail`, `GCPAuditLogs`	other clouds
`ThreatIntelligenceIndicator` (and `ThreatIntelIndicators`)	TI for `join` or the TI analytics rules
`Heartbeat`, `Usage`	agent health, ingestion volume (prove the data is arriving)

- **ASIM parsers** normalise across vendors: `_Im_ProcessCreate`, `_Im_NetworkSession`, `_Im_Dns`, `_Im_Authentication`, `_Im_WebSession`, `_Im_FileEvent`, `_Im_RegistryEvent`. Call with filters as parameters for speed: `_Im_Dns(starttime=ago(1d), domain_has_any=dynamic(["evil.test"]))`.
- **Watchlists**: `_GetWatchlist('tier0-assets') | project SearchKey` then `in` or `join`.
- **externaldata**: `externaldata(Ip:string) [@"https://.../list.csv"] with(format="csv")` pulls a public list at query time (rules cannot use authenticated URLs).

Performance and gotchas

- **Time filter first**, on the table's own time column; one `where` per table before any join.
- `has` matches whole terms split on non-alphanumerics: `has "svchost"` matches `C:\Windows\svchost.exe`, but `has "svc"` does not. Use `contains` only on already-filtered data.
- `search "term"` across all tables is expensive; use it to find *which* table, then query that table.
- `==` is case-sensitive: `AccountName == "admin"` misses `Admin`. Use `=~` or `tolower()` for names.

- Always cap `make_set` / `make_list`; always `summarize` before `join`; `materialize()` a `let` table used more than once; `toscalar()` to turn a one-value result into a constant.
- Dynamic access: `Props.Name` works for simple keys; `Props["Key With Spaces"]` otherwise; cast with `tostring()` before `==`.
- Defender XDR advanced hunting keeps **30 days** of raw events and caps results at **10,000 rows** (and 30 s to 10 min CPU); Sentinel retention is per workspace (interactive up to 2 years, archive/lake longer) and the portal shows up to 30,000 rows.
- `innerunique` is the default join: say `kind=inner` or `kind=leftouter` explicitly, every time.
- `dcount` is approximate by default; it is fine for stacking, use `dcount(x, 4)` or `count_distinct()` when the number itself matters.
- Analytics rules: no `render`, no `let` referencing other rules, `externaldata` only to public URLs, and a `take` is not a filter.

Related: query index (every pattern in KQL and SPL side by side), playbooks, Splunk SPL cheat sheet, Windows event IDs.

HUNT-SAGE
iamjammie/hunt-sage